

Plan de Rediseño UI -- Dr. Bike Sydney

Objetivo: Transformar la UI actual (monolítica, inconsistente, multi-version) en una experiencia mobile-first unificada según la especificación de producto del cliente.

Stack: HTML/CSS/JS vanilla + Supabase + Stripe (mismo stack, sin migrar a framework)

Ejecución: Persona no técnica usando Claude Code, sesión por sesión

Gap Analysis: UI Actual vs UI Objetivo

Dimension	UI Actual	UI Objetivo
Diseno	2 sistemas distintos (index.html claro, landing.html oscuro, index-redesign navy)	Unico: azul electrico #0A58CA sobre fondo oscuro
Flujo booking	Todo en una sola pagina con scroll	Step-by-step: Tipo > Fecha/Hora > Summary > Pago > Review
Mobile	4 versiones distintas, Eruda en prod	Una sola app mobile estructurada con navegacion inferior
Tracking	track.html separado con Leaflet	Integrado en la app mobile con mapa y datos del mecanico
Pago	Stripe Elements funcional pero sin diseno mobile	Formulario de tarjeta con Apple Pay y branding Stripe
Review	No existe como flujo dedicado	Pantalla con estrellas y comentario
Auth	Solo admin (client-side) y PIN de mecanico	Login de cliente con email/password
Bookings	No hay vista dedicada	My Bookings con tabs Upcoming / History

Arquitectura Meta

En vez de 6+ archivos HTML sueltos con estilos duplicados, el resultado sera:

```
dr-bike-sydney/  
  index.html      <- SPA principal del cliente (mobile-first)  
  mechanic.html   <- App del mecanico  
  admin.html      <- Panel de administracion  
  landing.html    <- Landing page SEO (se mantiene, diseno unificado)  
  css/  
    variables.css <- Design tokens compartidos (colores, tipografia, spacing)  
    main.css      <- Estilos de la app principal  
    landing.css   <- Estilos de landing (importa variables.css)  
  js/  
    app.js        <- Logica principal de la app cliente  
    router.js     <- Navegacion entre pantallas (hash-based SPA)  
    stripe.js     <- Logica de pagos Stripe
```

supabase.js<- Cliente y queries de Supabase

components.js<- Componentes reutilizables (navbar, cards, modals)

Resumen de Sesiones

Sesion	Tema	Entregable	Tiempo
1	Design System + Estructura base	variables.css, router.js, SPA shell	~45 min
2	Home + Booking (Parte 1)	Pantallas Home y Service Type	~45 min
3	Booking (Parte 2) + Summary	Pantallas Date/Time, Location, Summary	~40 min
4	Payment + Tracking	Pantallas Payment y Tracking con mapa	~50 min
5	Review + Auth + My Bookings	Pantallas Review, Login, My Bookings	~40 min
6	Unificacion + PWA + Admin polish	landing unificado, icons, manifests, admin adaptado	~40 min

Sesion 1: Design System y Estructura Base

Objetivo: Crear los cimientos del nuevo diseno: tokens de diseno compartidos, sistema de navegacion SPA, y shell de la app.

Prompt para Claude Code

Trabaja en el repo Dr.Bike-Sydney. Esta es la Sesion 1 del plan de rediseño UI. Vamos a crear la base del nuevo diseno mobile-first. NO modifiques los HTML existentes aun -- trabajamos en archivos nuevos.

1. CREAR css/variables.css CON LOS DESIGN TOKENS:

- Basado en esta especificacion de marca:
- Color primario: #0A58CA (azul electrico)
 - Fondo principal: #0F0F0F (negro/gris muy oscuro)
 - Fondo de tarjetas/secciones: #1A1A1A (gris oscuro)
 - Texto principal: #FFFFFF (blanco)
 - Texto secundario: #A0A0A0 (gris medio)
 - Borde sutil: #2A2A2A
 - Color exito: #22C55E (verde)
 - Color error: #EF4444 (rojo)
 - Color warning: #F59E0B (ambar)

Define variables CSS en :root con:

- Colores (--color-primary, --color-bg, --color-surface, --color-text,

- color-text-secondary, --color-border, --color-success, --color-error, --color-warning)
- Tipografia (--font-family: system-ui, --font-size-xs/sm/base/lg/xl/2xl/3xl con valores en rem)
- Espaciado (--space-1 al --space-8 en rem)
- Border radius (--radius-sm: 0.375rem, --radius-md: 0.5rem, --radius-lg: 0.75rem, --radius-full: 9999px)
- Sombras (--shadow-sm, --shadow-md, --shadow-lg)
- Transiciones (--transition-fast: 150ms ease, --transition-base: 250ms ease)

2. CREAR js/router.js - SISTEMA DE NAVEGACION SPA:

La app cliente usara hash-based routing para navegar entre pantallas sin recargar la pagina. Crea un router simple:

- Define las rutas: #home, #book-service, #service-summary, #payment, #tracking, #review, #login, #my-bookings, #profile
- Cada ruta tiene un render() que muestra/oculta la seccion correspondiente
- El router detecta cambios de hash (window.addEventListener('hashchange'))
- Guarda el estado de la navegacion actual
- Funcion navigate(route, params) para cambiar de pantalla con datos opcionales
- Animacion de transicion entre pantallas (slide de derecha a izquierda, 300ms, usando CSS transform)
- Exporta el router como variable global window.router

3. CREAR index.html - SPA SHELL:

Crea un NUEVO index.html (renombra el actual a index-legacy.html como backup) que sea el shell de la SPA. Debe ser minimalista (~100 lineas):

Estructura:

- <head>: meta viewport mobile-first, charset UTF-8, titulo "Dr. Bike Sydney", link a css/variables.css y css/main.css, link al manifest.json, meta theme-color #0F0F0F
- <body>:
 - <div id="app"> con todas las secciones/pantallas como divs ocultos (data-screen="home", data-screen="book-service", etc.)
 - Solo la pantalla activa tiene class="screen active"
- Scripts al final: js/router.js, js/supabase.js, js/components.js, js/app.js (en ese orden, type="module")
- NO incluyas contenido de las pantallas aun -- solo el shell vacio con los data-screen placeholders

4. CREAR js/components.js - COMPONENTES REUTILIZABLES:

Crea funciones que generen HTML para componentes comunes:

- createHeader(title, showBack, backUrl) - encabezado con titulo y boton back
- createBottomNav(activeTab) - barra de navegacion inferior con Home, Bookings, Track, Profile (iconos SVG inline simples)
- createButton(text, variant, fullWidth) - variant: 'primary' (azul), 'secondary' (borde blanco), 'ghost' (transparente)
- createServiceCard(service) - tarjeta de tipo de servicio

- createTimeSlot(time, available) - bloque horario seleccionable
- createStarRating(rating, interactive) - estrellas de calificación

Usa template literals. Los componentes deben usar las variables CSS de variables.css (no hardcodear colores).

5. CREAR css/main.css CON ESTILOS BASE:

- Reset CSS minimo (box-sizing, margin 0)
- Estilos base de body (bg: var(--color-bg), color: var(--color-text), font-family)
- Clase .screen { display: none; } y .screen.active { display: flex; flex-direction: column; min-height: 100dvh; }
- Padding global consistente: .screen { padding: 1rem; }
- Animacion de transicion entre pantallas:
@keyframes slideInRight { from { transform: translateX(100%); opacity: 0; }
to { transform: translateX(0); opacity: 1; } }
.screen.active { animation: slideInRight 300ms ease; }
- Estilos base para header, bottom nav, botones usando las variables CSS

Verificacion: Abre el NUEVO index.html en el navegador. Debe verse un fondo oscuro con texto blanco. index-legacy.html debe seguir existiendo como backup.

Verificacion

- ☐ css/variables.css existe con todos los tokens de disenio
- ☐ css/main.css existe con estilos base y animaciones
- ☐ js/router.js existe con sistema de navegacion hash-based
- ☐ js/components.js existe con funciones de componentes
- ☐ index.html nuevo existe como SPA shell (~100 lineas)
- ☐ index-legacy.html es el backup del index original
- ☐ Abrir index.html en navegador muestra fondo oscuro, sin errores en consola

Sesion 2: Pantallas Home y Book a Service (Parte 1)

Objetivo: Construir las dos primeras pantallas del flujo del cliente con datos reales de Supabase.

Prompt para Claude Code

Trabaja en el repo Dr.Bike-Sydney. Sesion 2 del plan de redisenio UI.
Vamos a construir las pantallas Home y Book a Service (seleccion de tipo de servicio).

1. CREAR js/supabase.js - CLIENTE SUPABASE:

- Inicializa el cliente de Supabase con las credenciales (usa las mismas que ya estan en el codigo existente -- buscalas en los archivos HTML actuales)
- Exporta funciones helper:
 - getServices() - obtiene lista de servicios desde tabla services
 - getAvailableSlots(date, serviceId) - obtiene slots disponibles para una fecha

- createBooking(bookingData) - crea un nuevo booking
- getMechanicInfo(mechanicId) - obtiene info del mecanico asignado
- getBookingStatus(bookingId) - obtiene estado actual del booking
- submitReview(bookingId, rating, comment) - envia una resena
- signIn(email, password) - login del cliente
- signUp(email, password, name) - registro del cliente
- getMyBookings() - obtiene bookings del cliente autenticado

NOTA: Si alguna tabla no existe en Supabase, la funcion debe devolver datos mock para que la UI funcione y no se rompa. Usa try/catch con fallback a datos de ejemplo.

2. CONSTRUIR PANTALLA HOME (data-screen="home"):

En index.html, dentro de <div data-screen="home">, construye esta estructura:

- HEADER: Logo "DR BIKE SYDNEY" en blanco, alineado a la izquierda
- HERO SECTION:
 - Titulo grande: "Professional bike service at your doorstep."
 - Subtitulo: "Mobile bicycle mechanic in Sydney."
 - Boton CTA primario: "Book a Service" (ancho completo, azul)
 - Boton secundario: "Call 0400 123 456" (borde blanco, ancho completo)
- SERVICE PILLARS (3 iconos en fila):
 - Icono + "Qualified Mechanic"
 - Icono + "Fully Equipped Mobile Workshop"
 - Icono + "5 Star Rated Service"
 - Usa SVG inline simples para los iconos (llave inglesa, camioneta, estrella)
- BOTTOM NAV: Home (activo), Bookings, Track, Profile

La pantalla Home NO debe tener scroll horizontal. Todo en una vista mobile (~390px de ancho maximo centrado).

3. CONSTRUIR PANTALLA BOOK A SERVICE (data-screen="book-service"):

- HEADER: Boton back (<-), titulo "Book a Service"
- SERVICE TYPE SECTION:
 - Titulo de seccion: "Service Type"
 - Cargar servicios desde Supabase via getServices()
 - Cada servicio es una tarjeta seleccionable (radio-button visual):
 - Nombre del servicio (ej. "General Service")
 - Precio desde (ej. "From \$120")
 - Icono o ilustracion a la izquierda (placeholder por ahora)
 - La tarjeta seleccionada tiene borde azul y fondo ligeramente azul
- Al seleccionar un servicio y un horario, guarda los datos en window.appState (objeto global: { service, date, time, location })
- El boton Continue navega a #service-summary

NOTA: Las secciones de Date y Time iran en la Sesion 3. Por ahora solo Service Type.

4. ESTILOS EN css/main.css:

Agrega estilos para:

- .hero-section con padding generoso, texto centrado

- `.btn-primary` (fondo: `--color-primary`, texto blanco, `border-radius`, `padding`, hover state con brillo)
- `.btn-secondary` (borde: `--color-text`, texto blanco, fondo transparente)
- `.service-card` (fondo: `--color-surface`, borde: `--color-border`, seleccionada: borde `--color-primary`, transicion)
- `.pillars-row` (flex row, iconos pequenos con texto debajo)
- `.bottom-nav` (fixed bottom, flex row, iconos con labels)

Verificacion:

- Al abrir `index.html`, se ve la pantalla Home con los 3 pilares y botones
- Al hacer click en "Book a Service", navega a `#book-service` y se ven las tarjetas de servicio
- Los datos vienen de Supabase (o del fallback mock si no hay tablas)
- Sin errores en consola

Verificacion

- ☐ `js/supabase.js` existe con todas las funciones helper (con fallback mock)
- ☐ Pantalla Home muestra hero, pilares, y botones CTA
- ☐ Pantalla Book a Service muestra tarjetas de servicio seleccionables
- ☐ Navegacion entre pantallas funciona (Home -> Book a Service -> Back)
- ☐ `window.appState` guarda el servicio seleccionado
- ☐ Estilos usan variables CSS de `variables.css`

Sesion 3: Book a Service (Parte 2) + Service Summary

Objetivo: Completar el flujo de reserva con seleccion de fecha, horario, ubicacion y pantalla de resumen.

Prompt para Claude Code

Trabaja en el repo Dr.Bike-Sydney. Sesion 3 del plan de rediseño UI.
Completamos el flujo de booking: Date, Time, Location y Service Summary.

1. COMPLETAR PANTALLA BOOK A SERVICE (data-screen="book-service"):

Agrega debajo de Service Type:

a) SELECT DATE SECTION:

- Titulo: "Select Date"
- Carrusel horizontal de dias (hoy + 6 dias hacia adelante)
- Cada dia muestra: nombre corto del dia (Mon, Tue, Wed...), numero del dia (20, 21, 22...), mes (May, Jun...)
- Formato de item: circulo/ovalito con dia arriba y fecha abajo
- Item seleccionado: fondo azul (`--color-primary`), texto blanco
- Items no seleccionados: fondo `--color-surface`, texto `--color-text-secondary`
- El carrusel debe ser scrolleable horizontalmente
- Al seleccionar un dia, actualiza `appState.date` y carga los horarios disponibles

b) SELECT TIME SECTION:

- Título: "Select Time"
- Grid de 2 columnas con slots horarios (8:00 AM, 10:00 AM, 12:00 PM, 2:00 PM, 4:00 PM, etc.)
- Cargar slots desde Supabase via `getAvailableSlots(date, serviceId)`
- Slot disponible: fondo `--color-surface`, borde `--color-border`
- Slot seleccionado: fondo `--color-primary`, texto blanco
- Slot no disponible: opacidad 0.4, no cliqueable, tachado sutil
- Al seleccionar, actualiza `appState.time`

c) LOCATION SECTION:

- Título: "Location"
- Muestra icono de casa + "Home" como ubicacion por defecto
- Boton "Change" que por ahora muestra un `alert("Location change coming soon")`
- Guarda la ubicacion en `appState.location`

d) BOTON CONTINUE:

- Solo habilitado si `appState.service AND appState.date AND appState.time` estan definidos
- Deshabilitado (opacidad 0.5) si falta algun campo
- Navega a `#service-summary`

2. CONSTRUIR PANTALLA SERVICE SUMMARY (data-screen="service-summary"):

- HEADER: Boton back, titulo "Service Summary"
- IMAGEN ILUSTRATIVA: Un placeholder visual del servicio (SVG de bicicleta estilizada o icono grande)
- DETALLES DEL SERVICIO (tarjeta con fondo `--color-surface`, padding, border-radius):
 - Service: "General Service" (o el seleccionado) -- \$150
 - Description: texto descriptivo del servicio desde Supabase
 - Date: fecha formateada (ej. "Tue, 21 May 2024")
 - Time: hora formateada (ej. "10:00 AM")
 - Location: ubicacion (ej. "Home")
- TOTAL: "\$150.00" en texto grande y bold
- BOTON: "Confirm Booking" (primario, ancho completo)
- Al confirmar:
 - Llama a `createBooking()` en `supabase.js`
 - Muestra un loading state en el boton ("Confirming...")
 - Si exito: navega a `#tracking` con el booking ID
 - Si error: muestra mensaje de error debajo del boton

3. ACTUALIZAR js/components.js:

Agrega funciones:

- `createDateItem(date, isSelected)` - genera el HTML de un item de fecha
- `createTimeSlot(time, isAvailable, isSelected)` - genera slot horario
- `createSummaryRow(label, value)` - fila de detalle en el summary

4. ACTUALIZAR js/app.js:

La funcion `init()` debe:

- Inicializar Supabase

- Configurar el router
- Renderizar la pantalla activa
- Configurar event listeners

Agrega logica para:

- Cargar dias del carrusel de fechas (generar array de 7 dias desde hoy)
- Manejar seleccion de dia y carga de horarios
- Manejar el flujo de confirmacion de booking

Verificacion: Flujo completo Home -> Book Service -> seleccionar tipo -> seleccionar fecha -> seleccionar hora -> Continue -> Service Summary -> Confirm Booking -> Tracking (aunque tracking no tiene contenido real aun).

Verificacion

- ☐ Carrusel de fechas muestra 7 dias y es scrolleable
- ☐ Grid de horarios carga slots desde Supabase (o mock)
- ☐ Boton Continue se habilita solo cuando todos los campos estan seleccionados
- ☐ Pantalla Service Summary muestra todos los detalles del booking
- ☐ Boton Confirm Booking llama a createBooking y navega a tracking
- ☐ Flujo end-to-end funciona sin errores de consola

Sesion 4: Payment y Tracking

Objetivo: Construir las pantallas de pago con Stripe y tracking con mapa en tiempo real.

Prompt para Claude Code

Trabaja en el repo Dr.Bike-Sydney. Sesion 4 del plan de redisenio UI.
Construimos Payment (Stripe) y Tracking (mapa en tiempo real).

1. CREAR js/stripe.js - LOGICA DE PAGOS:

- Inicializa Stripe con la publishable key (buscala en el codigo existente o en .env.example)
- Crea funciones:
 - initStripe() - inicializa Stripe.js
 - createPaymentForm(containerId) - renderiza Stripe Elements (cardNumber, expiry, cvc) dentro del contenedor
 - processPayment(amount, bookingId) - crea PaymentIntent via el backend y confirma el pago con Stripe
 - createPaymentRequestButton(containerId) - boton de Apple Pay / Google Pay

NOTA: El backend debe tener un endpoint /api/create-payment-intent.js.

Si no existe, crea uno simple que:

- Reciba amount y bookingId
- Cree un PaymentIntent en Stripe
- Devuelva el clientSecret

Usa las mismas credenciales de Stripe que ya estan en el proyecto

(busca STRIPE_SECRET_KEY en los archivos API existentes).

2. CONSTRUIR PANTALLA PAYMENT (data-screen="payment"):

- HEADER: Boton back, titulo "Payment", subtítulo "Service #DBS-XXXX"
- MONTO: "Total Amount" + "\$150.00" (o el monto del servicio seleccionado) en texto grande
- METODOS DE PAGO: Iconos/logos de VISA, Mastercard, Apple Pay en fila
- FORMULARIO DE TARJETA:
 - Un div con id="card-element" donde Stripe Elements renderiza el formulario
 - Estilo del formulario integrado con el tema oscuro:
 - Fondo: --color-surface
 - Borde: --color-border
 - Texto: --color-text
 - Placeholder: --color-text-secondary
- Payment Request Button (Apple Pay / Google Pay) encima del formulario
- BOTON: "Pay \$150.00" (primario, ancho completo)
- SEGURIDAD: Texto pequeño abajo: "Secure payment powered by Stripe. Encrypted and safe." con icono de candado
- Al pagar:
 - Boton muestra "Processing..."
 - Si éxito: navega a #tracking
 - Si error: muestra mensaje de error en rojo
 - Si se requiere autenticación 3D Secure: Stripe maneja el modal

3. CONSTRUIR PANTALLA TRACKING (data-screen="tracking"):

- HEADER: Boton back, titulo "Tracking", ID de servicio "#DBS-1024"
- STATUS INDICATOR:
 - Circulo verde pulsante + "In Progress - We're on the way!"
 - Los estados posibles son: confirmed, enroute, in_progress, completed
 - Cada estado tiene su color (verde para enroute, azul para confirmed, etc.)
- MAPA INTERACTIVO:
 - Usa Leaflet (ya esta en el proyecto) o Google Maps
 - Centrado en Sydney
 - Marker de la ubicación del cliente (casa, icono azul)
 - Marker del mecanico (camioneta, se actualiza en tiempo real)
- ESTIMATED ARRIVAL:
 - "Estimated Arrival: 10:15 AM"
 - Foto/avatar del mecanico + "John, your mechanic"
- BOTON: "Message Mechanic" (secundario, con icono de chat)
 - Por ahora, abre WhatsApp o muestra un toast "Chat coming soon"
- BOTTOM NAV: Home, Bookings, Track (activo), Profile

4. ACTUALIZAR js/supabase.js:

Agrega funcion:

- subscribeToMechanicLocation(bookingId, callback) - suscripcion realtime a la ubicación del mecanico
- Si la tabla no existe o la suscripcion falla, usa un mock que mueva el marker lentamente hacia la ubicación del cliente (para demo)

5. ESTILOS EN `css/main.css`:

- `#card-element`: estilos para Stripe Elements (usar las clases `.StripeElement`)
- `.status-indicator`: circulo con animacion de pulso
- `.map-container`: height 300px, border-radius, overflow hidden
- `.estimated-arrival`: tarjeta con avatar y datos del mecanico
- `.payment-methods`: fila de iconos de pago

Verificacion:

- Pantalla Payment muestra formulario de Stripe con el tema oscuro
- El boton de Apple Pay aparece si el navegador lo soporta
- Pantalla Tracking muestra mapa con markers
- Al completar booking, el flujo llega a tracking con datos reales

Verificacion

- ☐ `js/stripe.js` existe con inicializacion y formulario
- ☐ `api/create-payment-intent.js` existe (o se usa el endpoint existente)
- ☐ Pantalla Payment renderiza Stripe Elements con el tema oscuro
- ☐ Apple Pay / Google Pay aparecen como opcion
- ☐ Pantalla Tracking muestra mapa con marker del mecanico
- ☐ Indicador de estado con animacion de pulso
- ☐ Datos del mecanico y ETA visibles
- ☐ Flujo: Booking -> Payment -> Tracking completo sin errores

Sesion 5: Review, Auth y My Bookings

Objetivo: Construir pantallas de resena, login/registro, y lista de bookings.

Prompt para Claude Code

Trabaja en el repo Dr.Bike-Sydney. Sesion 5 del plan de rediseño UI.
Construimos Review, Login/Register, y My Bookings.

1. CONSTRUIR PANTALLA REVIEW SERVICE (data-screen="review"):

- HEADER: Boton back, titulo "Review Service"
- PREGUNTA: "How was your experience? We'd love to hear your feedback."
- ESTRELLAS INTERACTIVAS: 5 estrellas, al hacer hover/tap se iluminan en azul
 - Usa SVG de estrella, animacion de escala al seleccionar
 - Guarda el rating en `appState.review.rating`
- CAJA DE COMENTARIO:
 - Textarea con placeholder "Tell us about your experience..."
 - Fondo `--color-surface`, borde `--color-border`
 - Maximo 500 caracteres, contador de caracteres debajo
- BOTON: "Submit Review" (primario, ancho completo)
- LINK: "Maybe Later" (texto secundario, navega a `#home`)
- Al enviar:
 - Llama a `submitReview()` en `supabase.js`

- Muestra toast de éxito: "Thanks for your feedback!"
- Navega a #home

2. CONSTRUIR PANTALLA LOGIN (data-screen="login"):

- LOGO: "DR BIKE SYDNEY" centrado en la parte superior
- TITULO: "Welcome Back!" + subtítulo: "Login to your account"
- FORMULARIO:
 - Email input (tipo email, placeholder "hello@drbike.com.au")
 - Password input (tipo password, placeholder "Password", con botón toggle visibilidad - icono de ojo)
 - Link "Forgot Password?" (alineado a la derecha, por ahora muestra alert)
- BOTON: "Login" (primario, ancho completo)
- FOOTER: "Don't have an account? Sign up" -> cambia a modo registro
- MODO REGISTRO (misma pantalla, toggle):
 - Agrega campo "Full Name"
 - Boton: "Create Account"
 - Footer: "Already have an account? Sign in"
- Al hacer login/registro:
 - Llama a signIn/signup en supabase.js
 - Guarda sesión en localStorage
 - Navega a #home
 - Si error: muestra mensaje debajo del formulario

3. CONSTRUIR PANTALLA MY BOOKINGS (data-screen="my-bookings"):

- HEADER: Botón back, título "My Bookings"
- TABS: "Upcoming" (activo por defecto) | "History"
 - Estilo: subrayado azul en el tab activo, texto gris en inactivo
- LISTA UPCOMING:
 - Carga bookings activos desde getMyBookings() en supabase.js
 - Cada ítem es una tarjeta con:
 - Fecha (formato corto: "21 MAY")
 - Tipo de servicio y precio
 - Estado del booking (chip coloreado: "Confirmed", "En Route", "In Progress")
 - Flecha para ver detalle (>)
 - Si no hay bookings: estado vacío con icono y texto "No upcoming bookings"
- LISTA HISTORY:
 - Similar a upcoming pero con bookings completados/cancelados
 - Muestra rating si el usuario ya calificó
 - Botón "Book Again" en bookings completados
- BOTTOM NAV: Home, Bookings (activo), Track, Profile

4. ACTUALIZAR js/components.js:

Agrega:

- createBookingCard(booking) - tarjeta de booking para la lista
- createEmptyState(icon, title, subtitle) - estado vacío genérico
- createToast(message, type) - notificación toast (success/error)

5. ACTUALIZAR js/supabase.js:

Verifica que signIn, signUp y getMyBookings funcionen correctamente.
Si las tablas de auth no existen, el mock debe simular login exitoso y devolver bookings de ejemplo.

Verificacion:

- Pantalla Review: estrellas interactivas, textarea con contador, submit funcional
- Pantalla Login: formulario funcional, toggle signin/signup
- Pantalla My Bookings: tabs Upcoming/History, muestra bookings reales o mock
- Navegacion inferior funciona en todas las pantallas

Verificacion

- ☐ Review: 5 estrellas clickeables, textarea con contador, submit a Supabase
- ☐ Login: formulario email/password, toggle signup con nombre, persiste sesion
- ☐ My Bookings: tabs funcionales, lista de bookings desde Supabase, empty state
- ☐ Bottom nav consistente en pantallas de cliente

Sesion 6: Unificacion, PWA y Polish

Objetivo: Unificar landing page con el nuevo diseno, configurar PWA correctamente, adaptar admin y mecanico al estilo nuevo.

Prompt para Claude Code

Trabaja en el repo Dr.Bike-Sydney. Sesion 6 (final) del plan de rediseño UI.
Unificamos todo, PWA, y polish final.

1. UNIFICAR LANDING PAGE:

- Actualiza landing.html para usar el mismo design system:
 - Agrega `<link rel="stylesheet" href="css/variables.css">`
 - Actualiza los colores hardcodeados para usar variables CSS
 - Asegura que los botones y tarjetas usen las mismas clases (`.btn-primary`, `.btn-secondary`, `.card` del `main.css`)
- La landing debe sentirse parte del mismo producto, pero con layout mas amplio (no solo mobile)
- Conserva el SEO (structured data, meta tags, Open Graph)
- Agrega un CTA prominente que diga "Open App" y abra `index.html`

2. CONFIGURAR PWA CORRECTAMENTE:

- Actualiza `manifest.json`:
 - `name`: "Dr. Bike Sydney"
 - `short_name`: "Dr. Bike"
 - `description`, `icons`, `start_url`: "/"
 - `theme_color`: "#0F0F0F"
 - `background_color`: "#0F0F0F"
 - `display`: "standalone"
- Actualiza `sw.js`:
 - Agrega el NUEVO `index.html` y los nuevos CSS/JS a `STATIC_ASSETS`

- Cambia la estrategia de activate a limpieza selectiva (solo caches viejos, no los actuales)
- Agrega los nuevos archivos de css y js al cache
- Verifica que los iconos PWA existen:
 - icon-192.png, icon-512.png deben ser el logo de Dr. Bike
 - Si no existen iconos apropiados, genera SVGs optimos para PWA

3. ADAPTAR MECHANIC.HTML AL NUEVO DISEÑO:

- Agrega css/variables.css
- Actualiza colores para que coincidan con el design system
- Mejora el diseno del PIN input (mas moderno, consistente con el tema oscuro)
- La lista de jobs debe usar las mismas tarjetas que My Bookings
- KPI dashboard: mejora visual con las variables CSS
- Asegura que funcione como PWA independiente (tiene su propio manifest)

4. ADAPTAR ADMIN.HTML AL NUEVO DISEÑO:

- Agrega css/variables.css
- Actualiza sidebar, header, y tarjetas para usar el tema oscuro
- Los KPIs, tablas y graficos deben ser consistentes con el nuevo diseno
- Si admin.html es muy grande, extrae su CSS a css/admin.css (misma tecnica que Sesion 5 del plan de saneamiento)

5. POLISH Y DETALLES FINALES:

- Agrega transiciones suaves en todas las interacciones (hover, focus, active)
- Asegura que los toast/notificaciones usen el design system
- Verifica contraste de colores (texto secundario sobre fondo oscuro debe ser legible -- minimo 4.5:1 para texto normal)
- Agrega estado de carga (skeleton screens o spinners) mientras se cargan datos de Supabase
- Verifica que funcione en iOS Safari, Chrome Android, y Chrome Desktop (responsive)
- Agrega meta tag apple-mobile-web-app-capable para standalone en iOS
- Agrega meta tag apple-mobile-web-app-status-bar-style con content "black"

6. LIMPIEZA FINAL:

- Elimina index-legacy.html (ya tenemos el nuevo funcionando)
- Elimina cualquier archivo CSS/JS viejo que ya no se use
- Actualiza .gitignore si es necesario
- Verifica que no queden referencias a archivos eliminados

Verificacion final:

- Abrir landing.html: diseno unificado, CTA a la app
- Abrir index.html en movil: instalar como PWA, abrir en standalone
- Flujo completo: Home -> Book -> Date -> Time -> Summary -> Payment -> Tracking -> Review -> Home
- Login y My Bookings funcionales
- Admin y Mechanic con el nuevo tema oscuro

- Sin errores de consola en ninguna pagina
- PWA instalable con iconos correctos

Verificacion

- ☐ `landing.html` usa `variables.css` y tiene disenio unificado
- ☐ PWA: manifest correcto, service worker actualizado, iconos apropiados
- ☐ `mechanic.html` tiene el tema oscuro del design system
- ☐ `admin.html` tiene el tema oscuro del design system
- ☐ Transiciones y animaciones suaves en toda la app
- ☐ Skeleton/spinner durante carga de datos
- ☐ Funciona en Safari iOS, Chrome Android, Chrome Desktop
- ☐ `index-legacy.html` eliminado
- ☐ Sin archivos huérfanos o referencias rotas

Checklist Global de Cierre UI

Al finalizar las 6 sesiones de UI, verifica:

- ☐ Diseno unificado: todas las paginas usan `css/variables.css`
- ☐ Paleta consistente: azul electrico #0A58CA sobre fondo oscuro #0F0F0F
- ☐ Flujo de booking completo: 6 pantallas funcionando (Home, Service Type, Date/Time, Summary, Payment, Tracking)
- ☐ Pantallas secundarias: Review, Login/Register, My Bookings
- ☐ Stripe Elements con tema oscuro
- ☐ Apple Pay / Google Pay integrados
- ☐ Tracking con mapa en tiempo real
- ☐ PWA instalable con manifest, service worker e iconos
- ☐ Landing page unificada con SEO conservado
- ☐ Admin y Mechanic adaptados al design system
- ☐ Sin codigo muerto de las versiones anteriores de UI
- ☐ Responsive: mobile-first pero funcional en desktop
- ☐ Contraste de colores accesible (WCAG AA)

Notas para el Ejecutor

1. **Orden:** Las sesiones de UI son independientes del plan de saneamiento. Pueden ejecutarse en paralelo o secuencialmente.
2. **Branch:** Crea una branch separada: `git checkout -b redesign-ui`
3. **Backup:** La Sesion 1 renombra `index.html` a `index-legacy.html`. Siempre hay backup.
4. **Supabase:** Las funciones en `supabase.js` usan fallback a datos mock. La UI funciona aunque las tablas no esten listas.
5. **Stripe:** Necesitas las keys de Stripe configuradas en Vercel. Si no estan, el formulario no rompe pero muestra un mensaje claro.
6. **Testing:** Entre sesiones, siempre abre `index.html` en el navegador y recorre el flujo completo para detectar regresiones.